

Operating System Security

Computer Systems Security · LEI · University of Minho · 2025-26

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-04-21

Operating System Security: Lecture Overview

The OS as Security Enforcer: TCB, trust hierarchy and containment

Process Isolation and Memory Protection: virtual memory, W^X, ASLR and canaries

The Syscall Interface: kernel attack surface and seccomp filtering

Privilege, Capabilities, and Least Privilege: from setuid to Linux capabilities

Containers, Namespaces, and Sandboxing: namespaces, cgroups, MAC and escape paths

OS Hardening and the Limits of OS Security: sysctls, Secure Boot and OS limits

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review.

The OS as Security Enforcer

If the kernel is compromised, what does any application-level defence still guarantee?

Why OS Security Is the Foundation

- Most software vulnerabilities are contained or amplified by the OS
- The OS mediates every interaction: memory, files, network, devices
- A compromised process has only the rights the OS permits
- OS security determines the blast radius of software vulnerabilities
- **Containment layer:** OS mechanisms limit what a successful exploit can achieve, even when they cannot prevent it
- This module asks: what does the OS guarantee, and where do those guarantees fail?

Every other module (applications, crypto, network) runs on top of these OS guarantees.

The Trusted Computing Base (TCB)

- **TCB**: all hardware, firmware, and software whose correctness is required for security to hold
- Everything outside the TCB must trust the TCB; the TCB itself trusts nothing above it
- **Minimise the TCB**: less code means fewer vulnerabilities; smaller surface means easier verification
- Monolithic kernels have large TCBs; microkernels move services outside the TCB
- The Linux kernel contains roughly 27 million lines of code: all of it is TCB
- Principle: make the TCB as small and formally verifiable as possible

Every line of code inside the TCB must be treated as security-critical.

The Trust Hierarchy

- **Ring 0 (kernel mode)**: full hardware access; OS kernel executes here
- **Ring 3 (user mode)**: restricted; crossing to ring 0 requires an explicit syscall
- **Hypervisor (ring -1)**: manages multiple OS instances on one machine; trusted by the OS
- Firmware and hardware sit below the hypervisor and are trusted unconditionally by the OS
- Each layer trusts the layer below and is distrusted by the layer above
- A compromise at any layer undermines all assumptions made by layers above it

Trust hierarchy: Ring -1 (hypervisor) < Ring 0 (kernel) < Ring 3 (user)

OS Containment in Practice

- **Log4Shell (CVE-2021-44228)**: achieved remote code execution in the JVM; what the OS permitted determined the impact
- **Uncontained server**: attacker reads secrets, pivots to other hosts, persists via cron
- **Well-configured container**: attacker reaches a read-only filesystem, unreachable network, and no writable paths
- The same CVE is a catastrophe on a bare metal server and a manageable incident inside a well-scoped container
- Defence in depth requires each layer to limit the blast radius of failures in the layer above
- **TCB update**: *kernel, hypervisor, firmware and hardware are the trusted layers the OS relies on unconditionally.*

Uncontained server: full-system blast radius

Well-scoped container: bounded, manageable incident

Case Study: Pwnkit and the Expanding TCB

Background reading.

- **What happened:** January 2022, CVE-2021-4034: a decade-old memory corruption in PolKit's setuid pkexec granted any local user root on essentially every Linux distribution.
- **Which primitive failed:** Setuid is coarse-grained; the victim was a helper you inherited from the base install and probably never audited.
- **What the fix established:** Shrinking setuid binaries is part of TCB hygiene; modern distributions move to Polkit capability-based flows to eliminate setuid helpers wherever possible.

References: [Qualys advisory](#) | [NVD CVE-2021-4034](#)

Process Isolation and Memory Protection

What does a process believe about its own memory, and how does the hardware maintain that fiction?

Virtual Memory as an Isolation Primitive

- Each process has its own virtual address space: 0x0000... to 0x7FFF... on x86-64
- Virtual addresses are translated to physical addresses by the MMU on every memory access
- The page table is managed exclusively by the kernel; user space cannot modify it directly
- No process can directly read or write another process's memory: the MMU enforces this hardware-level isolation
- Kernel memory (above 0xFFFF8000_00000000 on x86-64) is mapped but inaccessible from user space
- Virtual memory is the architectural foundation of OS-level process isolation

Without virtual memory, privilege rings alone cannot keep processes apart.

Privilege Rings: User Mode and Kernel Mode

- x86 defines four privilege rings (0 through 3); Linux uses only ring 0 and ring 3
- **Ring 0 (kernel mode)**: can execute privileged instructions, access all physical memory, configure hardware
- **Ring 3 (user mode)**: cannot execute privileged instructions; attempting triggers a General Protection Fault
- System calls are the sanctioned crossing point: CPU switches from ring 3 to ring 0 and back
- The ring 3 to ring 0 transition is the most security-critical operation in the system
- A kernel bug at the boundary can allow ring 3 code to behave as if running in ring 0

Ring 3 (user, restricted) vs Ring 0 (kernel, privileged)

A single ring-boundary bug is worth thousands of ring-3 bugs to an attacker.

The W^X Policy: NX / DEP

- **W^X**: a memory page should be either Writable or eXecutable, never both simultaneously
- **NX (No-eXecute)**: a hardware bit in the page table entry marks pages non-executable; enforced by the MMU
- **DEP (Data Execution Prevention)**: the Windows marketing name for the same NX mechanism
- Prevents simple shellcode injection: the attacker controls data but cannot execute it directly
- **Bypass**: Return-Oriented Programming (ROP): chain existing executable code; no new code is injected
- W^X raises the sophistication required for code execution exploits without eliminating them

Defence: W^X (NX / DEP): pages may be Writable or eXecutable, never both

Bypass: Return-Oriented Programming (ROP): chain existing executable code

Address Space Layout Randomisation (ASLR)

- ASLR randomises the base address of each memory region at process load time
- **Randomised regions:** stack, heap, shared libraries, and the executable itself if compiled with PIE
- Without ASLR, exploit payloads can hardcode addresses; with ASLR, an address must be leaked first
- Information leaks (CWE-125, CWE-134) become a prerequisite for memory-corruption exploits when ASLR is active
- **Entropy:** modest on 32-bit systems (~16 bits, brute-forceable); much stronger on 64-bit (28 to 32 bits)
- **Bypass techniques:** information disclosure (leak a pointer), heap spraying, or brute-force on 32-bit platforms

Defence: ASLR randomises base addresses of stack, heap, libraries, PIE binaries

ASLR adds expected work, not a proof of safety: one pointer leak collapses the entropy.

Kernel ASLR (KASLR)

- KASLR randomises the base address of the kernel image in physical memory at boot time
- Without KASLR, kernel symbol addresses are fixed: any attacker who can read kernel memory knows every function pointer
- With KASLR, the attacker must first leak a kernel address before constructing a kernel exploit
- **Entropy**: modest (~9 bits on x86-64); supplemented by Kernel Address Space Layout Randomisation improvements in recent kernels
- **Bypass**: hardware side-channels (Spectre, Meltdown), kernel information leaks, or /proc/kallsyms if readable
- **Hardening**: set kptr_restrict=2 to hide kernel symbol addresses from all users, including root

Defence: KASLR randomises the kernel base address at boot

Even with kptr_restrict, side-channels read the layout: KASLR degrades gracefully, not absolutely.

Stack Canaries: The OS Perspective

- Stack canaries are a compiler mitigation; the OS participates in their randomisation at process start
- At process start, the kernel loads a fresh random value into the auxiliary vector (AT_RANDOM)
- The C library reads AT_RANDOM and places a derived value between local variables and the saved return address
- An overflowing write must pass through the canary; if the canary changes, the process is terminated
- The canary is fresh each process start, not each function call; process isolation prevents cross-process leakage
- **Bypass:** leak the canary value (e.g., via a format string vulnerability), or overflow a path that bypasses the canary (heap overflow)

Defence: stack canary placed between locals and return address; process-fresh at start

Canaries defend the return path: heap overflows and non-contiguous overwrites skip the check entirely.

Hardware Side-Channels: When Isolation Fails

- **Spectre (CVE-2017-5753, -5715)**: speculative execution reads memory out-of-order; cache timing traces remain observable by a sibling process
- **Meltdown (CVE-2017-5754)**: out-of-order execution briefly reads kernel memory before the privilege check raises an exception; cache retains the trace
- Both attacks violate the process isolation guarantee at the hardware level, not the software level
- **Mitigation**: Kernel Page-Table Isolation (KPTI): map the kernel only when executing in ring 0; incurs context-switch overhead
- Lesson: OS security guarantees are only as strong as the hardware faithfully implements them
- Spectre variants continue to be discovered; software mitigations are partial and ongoing
- **TCB update**: *the CPU's speculative and cached state is part of the TCB, and is not fully opaque to attackers.*

Hardware-level violation of OS isolation: software mitigations are partial and ongoing

Quick Check: Process Isolation and Memory

Self-test. Try to answer before moving on; the solution and reasoning are below.

A programme compiled with both NX and stack canaries enabled is exploited via ROP. Which assumption failed?

- (A)** Stack canaries detect all out-of-bounds writes on the stack.
- (B)** NX prevents execution of attacker-placed bytes, but does not prevent reuse of existing instructions.
- (C)** ASLR alone guarantees that no gadget address is predictable.
- (D)** ROP can only succeed when W^X is disabled.

Solution: A. Canaries only catch sequential overflows that pass through the canary slot. An attacker who leaks the canary, or writes via a non-sequential path (format strings, out-of-bounds-by-index), bypasses them. (B) is true, not a failed assumption; (C) and (D) overstate ASLR and misread W^X.

The Syscall Interface

Every useful program eventually asks the kernel for something. What does the kernel assume in reply?

System Calls: The Kernel's Public API

- System calls are the only sanctioned mechanism for user space to request kernel services
- Linux x86-64 defines roughly 350 system calls: read, write, open, socket, mmap, execve, clone, ...
- **Invocation:** load the syscall number into rax; arguments into rdi, rsi, rdx, r10, r8, r9; execute the syscall instruction
- The kernel validates the syscall number, checks argument ranges, and dispatches to the handler
- **Cost:** every syscall requires a full ring 3 to ring 0 to ring 3 round-trip; KPTI adds a TLB flush
- The syscall table is a fixed-size array indexed by syscall number; each entry is a function pointer

Linux x86-64: roughly 350 syscalls; every one is a Ring 3 to Ring 0 crossing

What Happens at a syscall Instruction

- The CPU saves user-space registers and the instruction pointer, then switches to the kernel stack
- The kernel dispatcher validates the syscall number and calls the handler
- **Pointer validation:** the handler must check that user-supplied pointers point into user space, not kernel memory
- **Permission check:** does the calling process hold the required capability or UID for this operation?
- The handler performs the requested operation and places the return value in rax
- The CPU restores user-space registers and resumes execution at the next instruction in ring 3

Argument Validation: A Persistent Challenge

- User-supplied pointers may reference kernel memory, I/O-mapped regions, or invalid addresses
- **copy_from_user / copy_to_user**: safe kernel helpers that verify the target range resides in user space before dereferencing
- **TOCTOU (CWE-362)**: kernel validates an address; user space changes the mapping during the window; the kernel then acts on a different address
- Integer arithmetic on user-supplied sizes can overflow, producing a smaller-than-expected allocation followed by an oversized copy
- Pointer arithmetic across struct fields can misalign access if size assumptions differ between userspace and kernel
- Argument validation errors at the syscall boundary are historically among the richest sources of kernel CVEs

The Kernel Attack Surface

- **Syscall handlers:** every system call is an entry point; roughly 350 on Linux
- **Loadable Kernel Modules (LKMs):** device drivers run in ring 0; a single bug in a driver is a full kernel vulnerability
- **/proc and /sys:** virtual filesystems expose kernel state; misconfigurations can leak sensitive information or allow privileged writes
- **Network protocol stacks:** packet parsing occurs in the kernel; a parsing error is a ring-0 vulnerability
- **eBPF:** allows user space to load verified programmes into the kernel; a bug in the verifier is a kernel code-execution vulnerability
- Minimising the kernel attack surface is a hardening principle: disable what you do not need

Seccomp: System Call Filtering

- **Seccomp**: Secure Computing Mode; a kernel facility that restricts which syscalls a process may invoke
- **SECCOMP_MODE_STRICT**: only read, write, exit, and sigreturn are permitted; used for single-purpose processes
- **SECCOMP_MODE_FILTER**: a BPF programme is evaluated on every syscall; can allow, deny, kill, or trap
- **Whitelist approach**: specify exactly which syscalls the application needs; deny all others
- **Violation outcome**: SIGKILL (immediate termination) or SIGSYS (trap for logging or graceful shutdown)
- Seccomp reduces the kernel attack surface to only the syscalls the application legitimately requires

Seccomp is the first syscall-layer defence-in-depth control: the kernel only considers what the profile allows.

Tracing Syscalls and Trialling seccomp

strace is the discovery tool; seccomp is the enforcement. Start with a real trace, then narrow it into an allow-list.

- **Count syscalls issued by a programme:** `strace -c -S ls /etc`
`%time calls syscall 34.18 12 close 22.95 8 mmap`
- **Trace a category and follow forks:** `strace -f -e trace=network ./client`
- **Derive an allow-list from the trace:** the syscall column becomes the SCMP_ACT_ALLOW set; everything else defaults to SCMP_ACT_ERRNO (EPERM).
- **Trial a custom profile on Docker:** `docker run --security-opt seccomp=./ls.json ubuntu ls /etc`

Rule: a deny-list chases new syscalls forever; an allow-list, built from the real trace, is what you ship to production.

Seccomp in Practice

- **Docker default profile:** blocks roughly 44 of 350 syscalls; examples: `reboot`, `kexec_load`, `perf_event_open`; `ptrace` is conditionally allowed when `CAP_SYS_PTRACE` is present
- **Chrome renderer sandbox:** restricts the renderer to a handful of I/O syscalls; cannot call `execve`, `fork`, or open arbitrary files
- **Android:** uses `seccomp-bpf` since Android 8.0; each application process carries a per-process policy
- **Writing a policy:** run `strace` on the application under realistic workload; whitelist observed syscalls; test edge cases
- **Lab connection:** *in Lab - Week 13 you will observe Docker's default seccomp profile blocking `unshare`, and the conditional `ptrace` allowance under `CAP_SYS_PTRACE`.*
- **TCB update:** *seccomp does not remove syscalls from the TCB; it removes them from this process's reachable attack surface.*

Every deployed server should ship a seccomp profile: a default-open kernel is the wrong baseline.

References: [Docker seccomp profile](#) | [Android seccomp-bpf](#)

Case Study: Dirty COW and the Syscall Attack Surface

Background reading.

- **What happened:** CVE-2016-5195: a race in copy-on-write handling let any local user write to read-only memory mappings, enabling tampering with suid binaries and root-owned files.
- **Which primitive failed:** Syscall permission checks ran before the commit; the race opened a window invisible to the validation logic.
- **What the fix established:** The kernel must hold a reference across the entire read-validate-write window; concurrency is part of the security contract, not an optimisation detail.

Privilege, Capabilities, and Least Privilege

Root is all-or-nothing. Is that the right unit of trust?

The Unix Privilege Model: Foundation

- Every process carries a real UID, effective UID, and saved set-UID (and the equivalent GID triplet)
- The kernel uses the effective UID for all permission checks: file access, signal sending, resource limits
- **uid=0 (root)**: bypasses nearly all Discretionary Access Control checks: file ownership, device access, capability grants
- The all-or-nothing nature of root is the core design problem of the Unix privilege model
- Setuid executables raise the effective UID to the file owner when executed: this mechanism extends the Unix privilege model from module 02
- Root-privileged programmes are high-value targets: a single vulnerability in them grants full system control

Setuid Revisited: A Design Smell

- **Intended use:** controlled privilege elevation; ping, passwd, and sudo rely on setuid
- **Problem:** the entire process runs elevated, not just the section that requires the additional privilege
- **File-descriptor leak:** open a privileged resource, drop privilege with `setuid(getuid())`, exec a shell; the file descriptor remains open
- Modern alternative: remove the setuid bit; grant only the specific Linux capability the programme needs
- Rule: if a programme needs one kernel privilege, grant that capability; never grant `uid=0`
- **Lab connection:** *in Lab - Week 13 you will compare granting a single Linux capability with running the same service as root, and test the boundary of `CAP_NET_BIND_SERVICE`.*

Rule: grant the specific capability a programme needs; never grant `uid=0`

References: [capabilities\(7\)](#) | [setuid\(2\)](#)

Linux Capabilities: Fine-Grained Privilege

- Linux capabilities decompose root's all-or-nothing power into roughly 40 independent privileges
- **CAP_NET_BIND_SERVICE**: bind to ports below 1024; allows a web server to run as a non-root user
- **CAP_DAC_OVERRIDE**: bypass file read, write, and execute permission checks for any file
- **CAP_SETUID**: make arbitrary setuid/setgid calls; if granted carelessly, allows re-gaining uid=0
- **CAP_SYS_ADMIN**: a broad catch-all; granting it is nearly equivalent to granting root
- Principle: grant the minimum set of capabilities required; audit with `getpcaps` and `capsh -print`
- **Runtime inspection**: read CapEff from `/proc/self/status` (hex bitmask); decode with `capsh -decode=<hex>` to list capability names

Capabilities are the primary mechanism for granting Linux programmes only the kernel powers they need.

Inspecting and Setting Capabilities

Commands assume an Ubuntu 22.04 / 24.04 host with libcap2-bin and util-linux installed; no container required.

- **Read file capabilities:** `getcap /usr/bin/ping`
`/usr/bin/ping cap_net_raw=ep`
- **Grant a capability (requires root):** `sudo setcap 'cap_net_bind_service+ep' ./my_http`
lets the binary bind to ports < 1024 without uid = 0.
- **Read a live process's capability sets:** `grep Cap /proc/self/status`
`CapInh: 0...0 CapPrm: 00001fff... CapEff: 00001fff... CapBnd: 00001fff...`
- **Decode a bitmask:** `capsh --decode=000001ffffffff`
returns the readable capability names.

Rule: if getcap returns nothing on a setuid binary, the binary still runs with full root authority: setuid root is the superset of every capability.

References: [capabilities\(7\)](#) | [setcap\(8\)](#)

Capability Sets: Bounding and Ambient

- Each process holds five capability sets: Permitted, Effective, Inheritable, Bounding, and Ambient
- **Permitted**: the maximum capabilities the process may place in its Effective set
- **Effective**: the capabilities currently active for permission checks
- **Bounding**: an upper bound on capabilities that can ever be raised; removing a capability from Bounding is permanent
- **Ambient**: capabilities automatically added to Effective and Permitted across execve, without requiring file capabilities
- Container runtimes drop dangerous capabilities from the Bounding set before starting the container init process

Five capability sets: Permitted . Effective . Inheritable . Bounding . Ambient

Container Runtimes and Capability Dropping

- By default, Docker containers start with roughly 14 of 40 capabilities
- **Dropped by default:** CAP_SYS_ADMIN, CAP_NET_ADMIN, CAP_SYS_PTRACE, CAP_SYS_MODULE, CAP_DAC_READ_SEARCH
- **Best practice:** docker run --cap-drop ALL --cap-add NET_BIND_SERVICE: grant only what the application requires
- **--privileged flag:** re-grants all capabilities and disables other security mechanisms; equivalent to running as root on the host
- Never run containers with --privileged or with CAP_SYS_ADMIN in production unless absolutely unavoidable
- **Lab connection:** *in Lab - Week 13 you will compare the attack surface of a fully privileged container against one that drops all Linux capabilities.*

```
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE: grant only what is needed
```

References: [Docker Security](#) | [runc capabilities](#)

Privilege Escalation: Connecting the Modules

- Local privilege escalation: a user-mode process exploits a kernel bug to gain root
- **Exploitation vector:** the kernel attack surface: syscall handlers, drivers, /proc, eBPF verifier
- The memory protection mechanisms (ASLR, KASLR, NX) must be bypassed before the kernel can be controlled
- **Typical chain:** information leak to defeat ASLR/KASLR, then ROP to bypass NX, then control-flow hijack in the kernel, then credential overwrite
- OS mitigations raise the cost of each step but do not eliminate the class of attack
- Kernel updates patch individual vulnerabilities; the architectural problem of a large TCB remains
- **TCB update:** *capabilities refine root within the policy; they do not shrink the kernel TCB that grants them.*

Case Study: Dirty Pipe and the Limits of Capabilities

Background reading.

- **What happened:** CVE-2022-0847 (Linux 5.8+): an uninitialised flag in pipe buffer structs let a user rewrite pages in read-only files, including /etc/passwd, without privilege.
- **Which primitive failed:** Neither DAC nor capabilities were involved: the bug bypassed the policy layer entirely by corrupting kernel-owned page cache.
- **What the fix established:** Capabilities refine privilege inside the policy; a kernel memory-safety bug ignores policy entirely. Kernel hardening (memory safety, KASAN in testing) remains a separate, non-negotiable layer.

Quick Check: Privilege and the Least-Privilege Model

Self-test. Try to answer before moving on; the solution and reasoning are below.

A service runs as UID 0 with cap-drop=ALL except CAP_NET_BIND_SERVICE. Which attacks does this still permit?

- (A)** The service can bind to low-numbered ports; but it cannot read /etc/shadow, load kernel modules, or change other users' files.
- (B)** No attacks are possible, because the process has dropped all capabilities except one.
- (C)** It can read arbitrary files, because root bypasses DAC regardless of capabilities.
- (D)** It can mount filesystems, because capability drops only affect networking.

Solution: C. UID 0 bypasses DAC permission checks regardless of capabilities, so the service can read /etc/shadow and most other root-owned files. Dropping capabilities removes privileged operations (kernel modules, raw networking, mount); it does not undo DAC bypass for the root UID. (A) and (B) underestimate the risk; (D) is wrong (mount requires CAP_SYS_ADMIN, unrelated to networking).

Containers, Namespaces, and Sandboxing

A container shares one kernel with its host. Is it isolated or merely partitioned?

Anatomy of a Container

- **Three restrictions, one workload:** a container is a set of one or more Linux processes running with a private view of system resources, bounded resource consumption, and reduced privilege
- **Restricted view (namespaces):** PID, network, mount, IPC, UTS, user, cgroup, and time namespaces each hide one resource type; Slide 39 unpacks the mechanism
- **Bounded resources (cgroups):** CPU, memory, and I/O quotas prevent a container from starving the host or its neighbours; Slide 42 unpacks the inspection commands
- **Reduced privilege:** capability drops (Slide 33), seccomp filters (Slide 23), and MAC profiles (Slide 48) compose to shrink what the kernel will do for the container
- **Not a virtual machine:** the container shares the host kernel with every other container; the alternative isolation boundary (a hypervisor) appears on Slide 47

A container = restricted view + bounded resources + reduced privilege; the kernel is still shared

What Linux Namespaces Isolate

- Namespaces virtualise a specific OS resource: processes inside a namespace see only their own view of that resource
- **Isolation vs security:** a namespace boundary provides isolation, not security by itself; security comes from combining namespaces with capability restrictions
- Container runtimes (Docker, podman, containerd) use namespaces to create the container abstraction
- **The /proc boundary:** /proc/version and kernel tunables are visible inside the container; they expose the shared kernel and cannot be hidden by any namespace type
- **Creation:** a process creates a new namespace with clone() or unshare() if it holds the appropriate capability
- **User namespaces:** allow unprivileged users to create isolated environments; they increase the kernel attack surface

A namespace boundary provides isolation, not security on its own

Namespaces are a building block: security comes from combining them with other mechanisms

Inspecting and Entering Linux Namespaces

Namespaces are exposed via /proc/<pid>/ns/ as magic symlinks. unshare and nsenter ship with util-linux.

- **Read your own namespace identity:** `ls -l /proc/self/ns/`
`mnt -> 'mnt:[4026531841]' net -> 'net:[4026531840]' pid -> 'pid:[4026531836]' ...`
- **Create a new network namespace:** `sudo unshare --net --fork bash`
inside the shell, ip link shows only lo.
- **Enter a running process's namespaces:** `sudo nsenter -t $(pidof sshd) -n ss -tlnp`
runs ss inside the target's network view.
- **Compare two processes:** `readlink /proc/1/ns/pid /proc/$$/ns/pid`
identical inode => same PID namespace.

Mental model: eight namespace references per process; if any inode differs, you live in a different world for that resource.

The Linux Namespace Types

- **PID**: processes see only PIDs within their own namespace; PID 1 inside the container is the container init
- **Mount**: processes have their own view of the filesystem hierarchy; enables the container root filesystem
- **Network**: processes have their own network interfaces, routing tables, and firewall rules
- **UTS**: processes have their own hostname and domain name
- **IPC**: processes have their own System V IPC objects and POSIX message queues
- **User**: processes have their own UID and GID mappings; unprivileged on the host maps to uid=0 inside the namespace

Six namespace types: PID . Mount . Network . UTS . IPC . User

cgroups: Not an Isolation Boundary

- **cgroups**: a kernel mechanism for limiting and accounting for resource usage across groups of processes
- **Controllable resources**: CPU time, memory, disk I/O, network bandwidth, and number of PIDs
- cgroups prevent resource exhaustion: fork bombs, memory pressure, and I/O saturation can be bounded per container
- Container runtimes use cgroups to enforce memory and CPU limits; security isolation is provided by namespaces and capabilities
- Combining namespaces, capabilities, cgroups, and seccomp is what makes a container; no single mechanism suffices

cgroups are NOT an isolation mechanism: they limit resources, not access

Containers combine namespaces, capabilities, cgroups, and seccomp: no single mechanism suffices

Inspecting cgroups (v2)

Ubuntu 22.04 runs cgroup v2 by default. Files live under /sys/fs/cgroup/; tools ship with systemd.

- **Read your own cgroup path:** `cat /proc/self/cgroup`
0::/user.slice/user-1000.slice/session-3.scope (single line, unified hierarchy)
- **See the live tree grouped by unit:** `systemd-cgls --no-pager`
- **Read an imposed memory limit:** `cat /sys/fs/cgroup/user.slice/user-1000.slice/memory.max`
returns 'max' if unbounded, or a byte count if limited.
- **Live usage per cgroup:** `systemd-cgtop`
CPU, memory, task count, refreshed like top.

Rule: cgroups answer "how much", not "who sees what". Resource limits alone are not isolation; combine with namespaces.

References: [cgroups\(7\)](#) | [systemd.resource-control\(5\)](#)

The Container Security Model

- A container is not a VM: all containers on a host share the same kernel
- A kernel vulnerability is exploitable from inside any container on the host
- **Container runtime:** a privileged daemon; a bug in Docker or containerd may allow container escape
- **Default Docker model:** user namespaces not enabled; container uid=0 maps to host uid=0
- **With user namespaces:** container uid=0 maps to an unprivileged host UID; reduces blast radius of container escape
- **Secure deployment:** drop capabilities, enable user namespaces, apply seccomp and AppArmor profiles, avoid `--privileged`

A kernel vulnerability is exploitable from inside any container on the host

Containers are only as strong as the policies the deployer applies: the default configuration is not secure.

Container Escape Techniques

- **Privileged container:** `--privileged` grants full capabilities; attacker can mount `/dev/sda` and write to the host filesystem
- **Docker socket mount:** if `/var/run/docker.sock` is bind-mounted into the container, the container can control the daemon and spawn privileged containers
- **CAP_SYS_ADMIN:** create a cgroup with `notify_on_release`; trigger an arbitrary command on the host
- **Kernel CVE:** a bug in a syscall handler is reachable from inside any container; escalate to host root via kernel exploit
- **Writable `/proc/sysrq-trigger`:** in a privileged container, writing to this file triggers host kernel actions
- **Defence:** read-only root filesystem, drop all capabilities, seccomp profile, AppArmor, network policies, and image scanning

Case Study: runc CVE-2019-5736 and the Shared Kernel

Background reading.

- **What happened:** A malicious container could overwrite the host's runc binary via /proc/self/exe; the next container exec then ran attacker-controlled code on the host as root.
- **Which primitive failed:** Containers share the host kernel and expose /proc paths referring to the runtime; the guest could reach the host's executable image through a kernel-provided symlink.
- **What the fix established:** Container escape is a host-kernel compromise; single-kernel multi-tenant models have a single-kernel blast radius. Mitigation requires copying runc to memfd before exec.

VM Hypervisors: A Stronger Isolation Boundary

- A hypervisor interposes on hardware access: each VM has its own virtual CPU, memory, and devices
- **Type 1 (bare metal)**: KVM, Xen, VMware ESXi run directly on hardware; smaller TCB than a full OS
- **Type 2 (hosted)**: VirtualBox, VMware Workstation run atop an OS; larger TCB and attack surface
- The VM escape attack surface is much smaller than a container escape: only the hypervisor API is exposed, not the full kernel syscall table
- **VM escape CVEs**: exist but are rarer: VENOM (CVE-2015-3456), speculative execution via shared physical CPU
- **Confidential computing**: AMD SEV and Intel TDX encrypt VM memory so the hypervisor cannot read guest data

Hypervisors: Type 1 (bare metal) vs Type 2 (hosted)

VM escape surface is much smaller than a container escape: only the hypervisor API is exposed

Linux Security Modules: AppArmor and SELinux

- **LSM (Linux Security Modules)**: kernel framework of hooks at security-relevant syscalls; AppArmor, SELinux, Yama, and Landlock plug in here
- **MAC**: Mandatory Access Control policies are defined by the administrator and cannot be overridden by the subject, including root
- **AppArmor**: path-based; profiles specify which files, capabilities, and network operations a programme may perform
- **SELinux**: label-based; every file, process, and object carries a security context; policy rules define allowed state transitions
- Both enforce MAC: even a root process is constrained by the policy if the policy is well-written and in enforcing mode
- **Docker default AppArmor profile**: blocks /proc/sysrq-trigger, /proc/kcore, and other dangerous writes
- **TCB update**: *the active LSM is part of the TCB; a policy bug is a privilege bug.*

MAC models: AppArmor (path-based) vs SELinux (label-based)

MAC is the backstop when every other control fails: a compromised process is still bound by the policy.

References: [AppArmor docs](#) | [SELinux project](#)

Quick Check: Containment Layers

Self-test. Try to answer before moving on; the solution and reasoning are below.

Given namespaces, cgroups, capabilities, seccomp, and MAC, which is the only one that is NOT an isolation mechanism?

- (A)** Namespaces: they partition the view each process has of kernel resources.
- (B)** Cgroups: they account and limit resource consumption but do not isolate visibility or authority.
- (C)** Capabilities: they subdivide root's authority.
- (D)** Seccomp: it filters the syscall surface a process can reach.

Solution: B. Cgroups account for and limit resource consumption (CPU, memory, I/O); they do not isolate visibility or authority. Namespaces partition what a process can see; capabilities and seccomp bound what it can do; MAC enforces who-can-touch-what. Only cgroups belong to a different category.

OS Hardening and the Limits of OS Security

Every defence so far has a bypass. What is the residual threat model after the hardening is complete?

OS Hardening Strategies

- **Attack surface reduction:** disable unused kernel modules (modprobe.d blacklists); remove unnecessary services and setuid binaries
- **Kernel hardening parameters:** tighten what the running kernel will reveal or permit via sysctl flags; the parameter menu follows on the next slide
- **Audit logging:** use auditd to record privilege escalation, setuid use, sensitive file access, and execve calls

Hardening: disable what is not needed, restrict what remains, log what matters

Kernel Hardening Parameters

- **kernel.randomize_va_space=2**: full ASLR; randomises stack, mmap base, and VDSO at process start
- **kernel.kptr_restrict=2**: hide kernel pointer values in /proc and dmesg, even from root
- **kernel.dmesg_restrict=1**: prevent unprivileged users from reading the kernel ring buffer
- **kernel.yama.ptrace_scope=1**: restrict ptrace to parent-child; blocks cross-process memory inspection by non-root
- **kernel.perf_event_paranoid=3**: restrict access to the perf subsystem; narrows a hardware side-channel vector
- **kernel.modules_disabled=1**: prevent loading further kernel modules after boot; irreversible until reboot
- **net.ipv4.conf.all.rp_filter=1**: enable reverse-path filtering; reject spoofed source addresses

Each hardening flag rejects an exploit class: their cost is near zero; their effect is cumulative.

References: [kernel.org admin-guide: sysctl](#)

Secure Boot and the Trusted Platform Module

- **Secure Boot:** UEFI verifies each stage of the boot chain using cryptographic signatures: firmware, bootloader, kernel
- An unsigned or modified kernel is rejected; protects against bootkit attacks and persistent kernel implants
- **TPM:** a tamper-resistant hardware chip that stores cryptographic keys and records boot measurements
- **PCRs:** Platform Configuration Registers; the TPM records a hash chain of every boot-stage measurement; any change invalidates the PCR values
- **Remote attestation:** a verifier can ask the TPM to prove the machine booted into a specific, known-good configuration

Chain of trust: UEFI verifies firmware + bootloader + kernel; TPM attests to the boot measurement

What OS Security Cannot Defend Against

Scope: each item below sits outside what the OS can see or enforce.

- **Kernel zero-days:** if the kernel contains an unknown vulnerability, all OS-level controls can be bypassed
- **Supply-chain compromise:** a malicious package inserted upstream arrives as legitimate code; the OS sees nothing wrong
- **Insider threats:** a user with legitimate root access can disable any OS control
- **Hardware vulnerabilities:** Spectre, Meltdown, and Rowhammer bypass software isolation at the hardware level
- **Configuration errors:** a misconfigured container, a permissive sudo rule, or a world-writable directory undoes all other hardening
- **Social engineering:** an attacker who convinces an administrator to run a malicious command bypasses all technical controls

Synthesis: Defences vs Attack Classes

Attack class	W^X	ASLR	Canaries	Seccomp	Capabilities	Namespaces	MAC	Secure Boot
ROP	◐	●	◐					
Kernel memory-safety bug				◐			◐	
Setuid misuse					●		●	
Container escape				●	●	◐	●	
Supply-chain compromise								◐
Hardware side-channel								
Social engineering								

● *substantially mitigates* ◐ *reduces probability or severity* (empty) *does not apply*

Defence in Depth: The Complete Stack

- **Software layer:** secure coding (CWE avoidance), static analysis, fuzzing, dependency auditing
- **OS process isolation:** virtual memory, privilege rings, ASLR, NX/DEP, stack canaries
- **OS containment:** Linux namespaces, capabilities, seccomp, AppArmor or SELinux profiles
- **Infrastructure:** network segmentation, firewall, IDS, centralised logging and alerting
- **Hardware:** Secure Boot, TPM, hardware memory tagging (ARM MTE), confidential computing
- No single layer is sufficient; compromise of any layer is limited by the layers below

Layers: Software . OS isolation . OS containment . Infrastructure . Hardware

Key Takeaways

- The OS is the foundation of system security: every software vulnerability's impact is bounded by OS controls
- Process isolation (virtual memory, privilege rings) is fundamental but has hardware-level limits, as Spectre and Meltdown demonstrate
- The syscall interface is the primary kernel attack surface; seccomp reduces it to what the application legitimately needs
- Linux capabilities replace the all-or-nothing root model with fine-grained privilege; setuid is a legacy mechanism
- Containers are not VMs: they share the kernel; container security requires namespaces, capabilities, seccomp, and MAC working together
- Defence in depth requires each OS layer to limit the blast radius of failures in the layer above